

00_frontmatter

Helios — Claude Code Operating Manual

`CLAUDE_CODE_WORKFLOW.md` · v1.0 · 2026-06-03 · *How to build Helios efficiently with Claude Code on Claude Pro, one developer, without hallucination.*

This manual is the *how-to-drive-the-tool* layer. `IMPLEMENTATION_PLAYBOOK.md` says **what** to build and **why**; this says **how to get Claude Code to produce it correctly**. The decisive fact: **the code for every milestone already lives in the repo docs** — so Claude Code's job is to *wire, adapt, and test* it, not invent it. Hand it the source of truth and it cannot hallucinate; let it guess and it will. Build to the lean scope (`LEAN_SCOPE.md`): **[Wk1-MVP]** M0–M5 + the decomposition, **[Wk2-v1]** one MCP server + one grounded LLM brief + a small honest eval, **[v2-later]** the rest.

Before you start: configure Claude Code for this repo

1. **Move `CLAUDE.md` to the repo root.** It currently sits in `docs/`. At the root it **auto-loads into every Claude Code session**, pinning the canonical names, conventions, and grounding rules — this is your single most important hallucination firewall. (*Also fix the two known doc drifts so Claude reads truth: the registry is `semantic_layer.yaml`, not `semantic_models.yaml`; the benchmark has 50 scenarios, not 34.*)
2. **Register the MCP server** (from M6) in `.mcp.json` / `mcp_servers.yaml` so Claude Code can only reach data through governed tools.
3. **Create the project slash commands** (`.claude/commands/`): `/resume`, `/new-model`, `/new-metric`, `/new-mcp-tool`, `/review-sql`, `/new-scenario`. They encode the prompt anatomy + review rubric so you don't retype them.
4. **One milestone per session.** `/clear` (or a fresh session) between milestones — no context bleed, lower usage, less drift.

The operating doctrine (9 habits that prevent hallucination)

- **D1 — Root** `CLAUDE.md` **is law**. Canonical names auto-load; Claude can't drift on `session_key`, `reached_*`, the 10 channel groups, the grains.
- **D2 — Attach the source-of-truth section** for every file. Claude *copies/adapts*; it doesn't guess.
- **D3 — Attach the real upstream files** (the actual columns it must reference) so it physically can't invent columns.
- **D4 — Demand citations**: "cite the exact doc section you used for each block."
- **D5 — Forbid invention**: "use ONLY names in the attached registry / GA4 schema; if a name isn't there, STOP and ASK — don't guess."
- **D6 — Let the machine catch it**: `dbt parse / compile` and BigQuery `dry_run` fail on non-existent columns; tests fail on wrong logic. Compile + dry-run + test *before* accepting.
- **D7 — One name source**: `semantic_layer.yaml` + the GA4 schema are the only places metric/column names come from — keep them attached.
- **D8 — /clear** **between milestones**.
- **D9 — Plan mode first** for any multi-file milestone — review the plan against the spec before a line of code exists.

The prompt anatomy (every codegen prompt)

```
[ROLE]      Implement Helios milestone M<n>, file <path>.
[SOURCE]    Copy/adapt the code in @<DOC $X>. Follow CLAUDE.md (auto-loaded).
[UPSTREAM]   Reference columns/refs ONLY from these: @<upstream files>.
[TASK]       <one concrete file/behavior>.
[CONSTRAINTS] Use ONLY canonical names from the semantic layer / GA4 schema.
              Do NOT invent columns, metrics, or tables. If a name isn't in the
              attached sources, STOP and ASK.
[TEST-FIRST] Write the test (<test>) first, then the code, then run <command>
              and show me the output.
[OUTPUT]     Cite the doc section used for each block.
```

The review rubric (never accept code that fails any of R1–R6)

	Check
R1	It cites the spec section it used.
R2	It parses/compiles (<code>dbt parse / compile</code> , <code>python import</code>) — no ref errors.
R3	Claude ran the milestone's test and showed it pass (don't take its word — see the output).
R4	Every column/metric name is canonical (in the registry / GA4 schema).

	Check
R5	The diff matches conventions (naming, materialization, <code>session_key</code> , <code>reached_*</code> , <code>money</code> , <code>grains</code>) — <i>you read it</i> .
R6	For SQL: dry_run clean (catches hallucinated columns + cost).

Claude Pro budget rules

- Plan mode is cheap; bad regenerated code is expensive — **plan before coding**.
- `/clear` between milestones (smaller context = less usage + less drift).
- Batch one milestone's related files in **one focused session**.
- **Don't burn budget running the product's LLM brief/agents repeatedly during dev.**
Build and test the *deterministic* layers (dbt, stats, the MCP server) with Claude Code; spend the LLM budget only when validating the M7/v1 brief.
- Subagents multiply usage — reserve them (e.g., a single code-review subagent), don't use them for everything.

Session opener (copy-paste)

```
We're building Helios milestone M<n> (<name>). CLAUDE.md is loaded.
I'm attaching @<source-doc $X> as the source of truth and @<upstream files>.
PLAN FIRST — don't write code yet: list the files, the tests to write first, and
confirm you'll use only canonical names from the attached registry/GA4 schema.
Then wait for my go.
```

How to read the per-milestone sections

Each milestone below has five parts — **Context to load** · **Files to provide** · **Prompts to use** · **Preventing hallucinations** · **Reviewing generated code** — and a lean tag: **[Wk1-MVP]**, **[Wk2-v1]**, or **[v2-later]**. Build the tagged-for-now milestones; the `[v2-later]` ones include how you'd drive Claude Code to build them, but they're out of the 2-week / Pro budget.